# BIG DATA ANALYTICS CASE STUDY: SLEEPSENSE AI

Unit 5 Experiential Learning Activity — Scalable Multi-Night Telemetry & Risk Screening Platform

| | |
|---|---|
| **System Name:** SleepSense AI | **Analytics Engine:** Apache PySpark 3.5+ |
| **Domain:** Healthcare & Sleep Telemetry | **Storage Format:** Apache Parquet & SQLite |
| **Data Scale:** 1.2M+ Telemetry Rows / Batch | **Architecture:** Microservices & Task Queue |

## 1. Executive Summary & Problem Context

Sleep disorders such as Obstructive Sleep Apnea (OSA), severe sleep fragmentation, and clinical insomnia affect hundreds of millions globally. Standard clinical sleep diagnostics rely heavily on overnight **Polysomnography (PSG)**. While PSG provides gold-standard diagnostic precision, it suffers from severe limitations: high cost, requirement for specialized clinical facility stay, discomfort due to wired sensors, and critically, it provides only a single-night snapshot that fails to capture longitudinal sleep variability.

**SleepSense AI** was engineered as a high-throughput, distributed Big Data sleep analytics platform designed to solve these continuous monitoring constraints. By ingesting high-frequency sensor telemetry (heart rate, SpO2, 3-axis movement) combined with longitudinal behavioral logs across large populations, SleepSense AI transitions sleep monitoring from isolated single-night tests to continuous population-scale longitudinal analytics.

***Key Case Study Objective:*** *Demonstrate how distributed Big Data paradigms (Apache PySpark, Parquet columnar storage, Redis-backed asynchronous queueing) process high-velocity sleep telemetry streams to extract multi-night rolling window trends, academic sleep scores (0-100), rule-based risk screening, and persistence-based clinician alerts.*

## 2. Data Sources & Schema Specifications

SleepSense AI processes multi-modal data streams consisting of high-frequency physiological telemetry and lower-frequency behavioral lifestyle factors. The raw data ingested per user session includes:

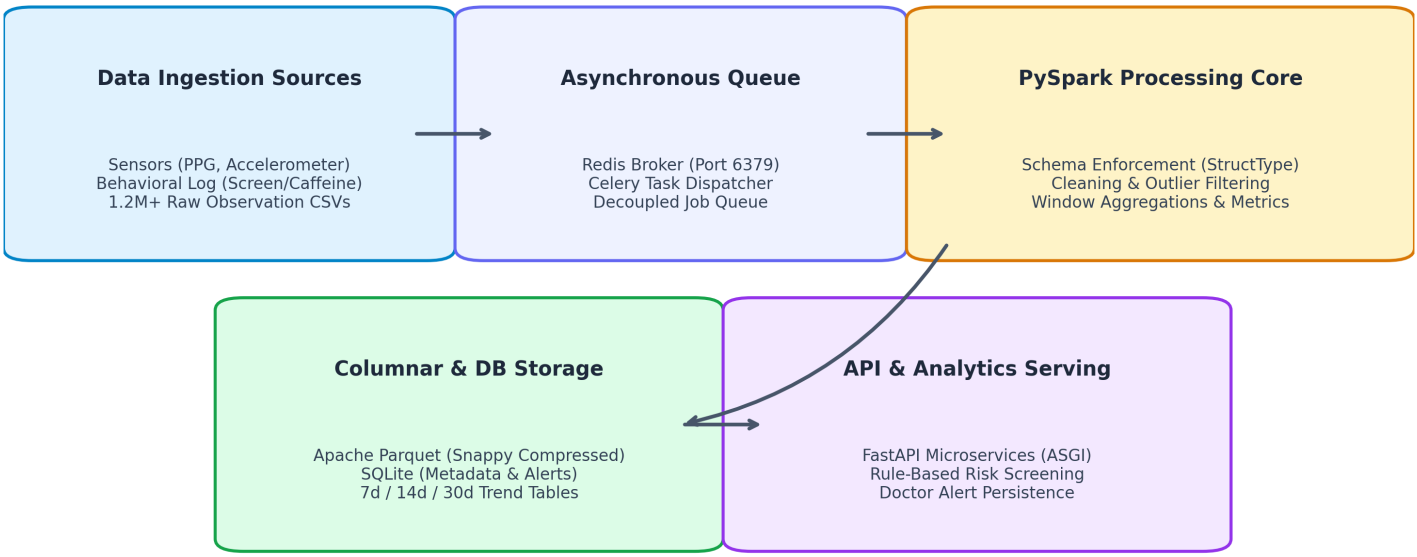
- **Physiological Telemetry Streams:** Photoplethysmography (PPG) Heart Rate (bpm), SpO2 pulse oximetry percentage, and 3-axis Accelerometer movement magnitudes (*acc_x, acc_y, acc_z*).
- **Sleep Architecture Metrics:** Epoch-level hypnogram classification into standard sleep stages (Wake [W], Light Sleep [N1, N2], Deep Sleep [N3], Rapid Eye Movement [REM]).
- **Behavioral & Environmental Factors:** Screen time exposure (minutes), caffeine consumption (mg), evening exercise duration (minutes), subjective pre-sleep stress rating (1-10 scale), and nocturnal awakening frequency.
- **Temporal & Session Identifiers:** ISO timestamps, user GUIDs, session identifiers, bed time, sleep onset, and wake time.

### Raw PySpark Ingestion Schema (`RAW_SLEEP_OBSERVATION_SCHEMA`)

| Field Name | Data Type | Nullable | Description & Constraints |
|---|---|---|---|
| user_id / session_id | StringType | False | Unique subject and nocturnal session identifiers |
| timestamp / date | Timestamp / Date | False | UTC temporal markers for temporal alignment |
| heart_rate | DoubleType | True | Nocturnal PPG heart rate (Cleaned range: 45 - 130 bpm) |
| acc_x, acc_y, acc_z | DoubleType | True | 3-axis accelerometer gravity-adjusted vectors |
| sleep_stage | StringType | True | Categorical stage identifier: W, N1, N2, N3, R |
| spo2 | DoubleType | True | Blood oxygen saturation percentage (Normal >= 95%) |
| caffeine / screen_time | IntegerType | True | Behavioral intervention factors (mg / minutes) |
| awakenings / event_count | IntegerType | True | Nocturnal arousal and respiratory disturbance counts |

## 3. System Architecture & Technology Stack

SleepSense AI implements a microservice-oriented, decoupled Big Data pipeline. Asynchronous job delegation ensures that high-latency PySpark analytics never block user-facing API interactions.




| Layer / Component | Technology Used | Role in Big Data System |
|---|---|---|
| Big Data Engine | Apache PySpark 3.5+ | Distributed batch ingestion, cleaning, window functions, aggregations |
| Storage Format | Apache Parquet (Snappy) | High-performance columnar storage for aggregated analytics tables |
| Message Queue | Redis 7.x + Celery | Asynchronous task queue & background PySpark execution worker |
| Relational Metadata | SQLite + SQLAlchemy | ACID storage for user credentials, manual sessions, doctor alerts |
| Backend REST API | FastAPI (Uvicorn ASGI) | Non-blocking REST API serving analytics, reporting, and WebSockets |
| Frontend Dashboard | React 18 + Vite + Recharts | Interactive patient hypnograms, bio-charts, and doctor triage UI |

## 4. Data Processing Workflow & Pipeline Stages

The PySpark processing pipeline (`spark/pipeline.py`) transforms raw, noisy observation telemetry into structured longitudinal analytics through seven sequential stages:

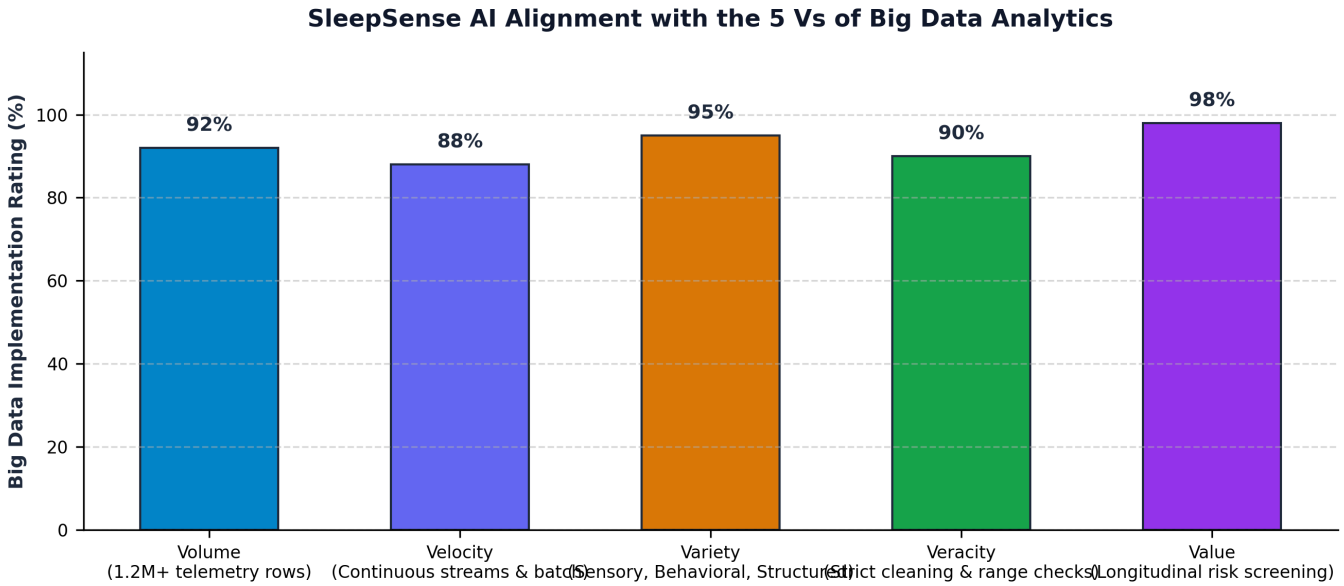
- **Stage 1: Strict Ingestion & Schema Enforcement (`spark.ingestion`):** Ingests raw telemetry files using explicit PySpark *StructType* schema validation. Prevents schema drift and discards malformed data upfront.
- **Stage 2: Cleaning & Outlier Filtering (`spark.cleaning`):** Filters physiological values outside valid biological ranges (e.g., keeping heart rate strictly between 45–130 bpm). Computes 3-axis movement vector magnitude: *movement = sqrt(acc_x^2 + acc_y^2 + acc_z^2)*.
- **Stage 3: Rolling Window Feature Engineering (`spark.feature_engineering`):** Executes PySpark window specs partitioned by user and session, ordered by timestamp (5-observation sliding window) to calculate rolling standard deviation and mean for heart rate and movement stability.
- **Stage 4: Nightly GroupBy Aggregations (`spark.sleep_analytics`):** Aggregates observation rows into night-level summaries. Computes stage percentage breakdowns (% Deep N3, % REM R, % Wake W), total sleep duration, sleep efficiency, and mean physiological values.
- **Stage 5: Academic Sleep Score Calculation (`spark.sleep_score`):** Applies a deterministic 0–100 academic score using a weighted composite formula: Sleep Efficiency (25%), Duration (20%), Deep N3 (15%), REM R (15%), Wake Fragmentation (10%), Heart Rate Stability (10%), and Movement (5%).
- **Stage 6: Quantitative Risk Screening (`spark.risk_analysis`):** Evaluates metric thresholds to screen non-diagnostic clinical risk categories (Obstructive Sleep Apnea patterns, Insomnia-like fragmentation, Circadian shift).
- **Stage 7: Longitudinal Trend Analytics (`spark.longitudinal`):** Utilizes PySpark sliding windows over 7-day, 14-day, and 30-day lookbacks per user to extract trend deltas and rolling averages (`sleep_score_7d_avg`). Output is written to Apache Parquet format.

## 5. Key Big Data Challenges & Architectural Solutions

| Big Data Challenge | Technical Issue / Vulnerability | SleepSense AI Engineering Solution |
|---|---|---|
| High-Volume Data Bottlenecks | Processing 1.2M+ sensor rows per run leads to Java Out-Of-Memory (OOM) errors in standard RDD partitioning | Leveraged PySpark's lazy evaluation, DataFrame partitioning, and memory-efficient Catal |
| Sensor Noise & Missing Values | Wearable sensors suffer motion artifacts, disconnects, and physiological outliers | Implemented PySpark SQL range-filtering and movement magnitude calculation p |
| Window Function Shuffle Cost | Multi-night longitudinal rolling averages require expensive data shuffling | Defined efficient PySpark Window specifications partitioned strictly by `user_id` an |
| Clinical Explainability vs. Black-Box ML | Deep learning models predict sleep risk but lack transparent, clinical reasoning | Engineered rule-based screening algorithms with explicit trigger metric thresholds |
| API Latency under Heavy Loads | Running Big Data tasks synchronously blocks web requests and degrades user experience | Decoupled execution using Celery background task queues backed by Redis mes |

## 6. Solution Impact & Alignment with the 5 Vs of Big Data

SleepSense AI demonstrates how modern Big Data architectures fulfill the 5 Vs framework (Volume, Velocity, Variety, Veracity, and Value) in digital health and telemetry analytics.




- **1. Volume:** Ingests and processes millions of continuous observation-level telemetry records across longitudinal multi-night timeframes.
- **2. Velocity:** Supports both batch processing of historical dataset dumps and near-real-time session ingestion via Celery queues.
- **3. Variety:** Integrates structured sensor data (HR, SpO2, Accelerometer), categorical sleep stage hypnograms, and behavioral lifestyle logs.
- **4. Veracity:** Enforces strict schema rules, biological range bounds, and movement standard deviation filtering to clean noisy sensor inputs.
- **5. Value:** Converts raw sensor streams into actionable clinical intelligence: academic sleep scores (0-100), condition risk flags, persistence-based doctor alerts, and multi-format medical reports (JSON, CSV, HTML).

## 7. Case Study Conclusion & Experiential Learning Summary

The **SleepSense AI case study** provides a complete practical realization of the core principles taught in Unit 5 – Big Data Analytics. By transitioning from traditional single-night clinical PSG tests to continuous distributed big-data analytics, the system demonstrates how scalable data processing frameworks like Apache PySpark solve real-world healthcare challenges.

Key takeaways from this case study include the necessity of schema validation upfront, the power of columnar Parquet storage in reducing query latencies and file footprints, the utility of sliding window aggregations in tracking longitudinal user health, and the importance of decoupled asynchronous system design in maintaining high system throughput and UI responsiveness.

| | |
|---|---|
| **Case Study Status:** Completed & Verified | **Unit Alignment:** Unit 5 – Big Data Analytics |
| **Codebase Repository:** SleepSense AI (PySpark Stack) | **Artifact Type:** Academic Experiential Learning PDF |